

Переменные и типы данных

Уверенно хранить текст, числа и логические значения в переменных и осознанно преобразовывать типы.

DAY 08

VARIABLE

int

float

str

bool

ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Уверенно хранить текст, числа и логические значения в переменных и осознанно преобразовывать типы.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Python и рабочее окружение

Что уже есть в нашей модели.



СЕГОДНЯ

Переменные и типы данных

Вчера: Python и рабочее окружение. Сегодня мы добавляем следующий слой: переменные и типы данных.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Почти любая программа начинается с данных. Если не понимать типы, ошибки вроде «можно ли сложить строку и число?» будут постоянно удивлять.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Переменная — подписанная коробка.
Имя говорит, что лежит внутри,
значение — содержимое, а тип
определяет, какие действия с ним
разумны.

ПЕРЕВОД В IT

variable	имя, связанное со значением в программе
int	тип целых чисел
float	тип чисел с дробной частью

ТЕОРИЯ

Слой 1: variable • int • float

Сначала смысл, затем профессиональный термин.

**variable**

имя, связанное со значением в программе

**int**

тип целых чисел

**float**

тип чисел с дробной частью

Слой 2: str • bool • None

Сначала смысл, затем профессиональный термин.

**str**

тип текстовых строк

**bool**

логический тип со значениями True и False

**None**

специальное значение отсутствия данных

Слой 3: type • assignment

Сначала смысл, затем профессиональный термин.

type

категория данных и допустимых операций

assignment

сохранение значения в переменной

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: constant • casting

Сначала смысл, затем профессиональный термин.

constant

значение, которое договорились не изменять

casting

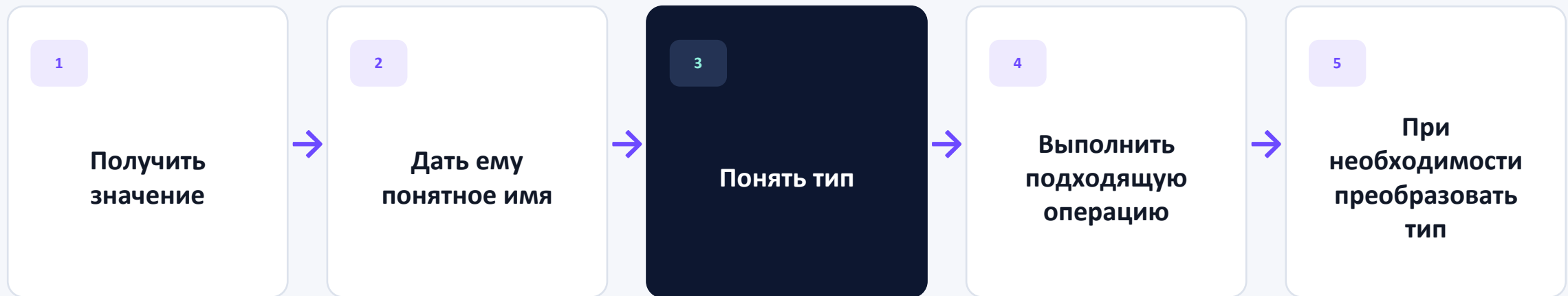
явное преобразование значения к другому типу

Не заучивай формулировку. Придумай свой пример использования каждого термина.

СХЕМА

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day08_step1

```
name = "Анна"  
age = 25  
is_beginner = True  
  
print(name)  
print(age)  
print(is_beginner)
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
name = "Анна"
```

Сохраняем значение или результат вычисления под понятным именем.

```
age = 25
```

Сохраняем значение или результат вычисления под понятным именем.

```
is_beginner = True
```

Сохраняем значение или результат вычисления под понятным именем.

```
print(name)
```

Показываем результат, чтобы увидеть состояние программы.

```
print(age)
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
name = "Анна"  
age = 25  
is_beginner = True  
  
print(name)  
print(age)  
print(is_beginner)
```

РЕЗУЛЬТАТ

Анна
25
True

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
price = 100  
price = price + 20  
print(price)
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

120

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

ОШИБКА

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



wrong

```
age = "25"  
print(age + 1)
```

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

"25" — строка, а 1 — число. Python не складывает их напрямую. Сначала строку нужно преобразовать в int.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.

```
age = "25"  
print(int(age) + 1)
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Посчитать стоимость покупки по цене и количеству, а затем проверить, дорогая ли получилась покупка.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Сохраняем цену числом.**
- 2 Сохраняем количество числом.
- 3 Умножаем цену на количество.
- 4 Сохраняем итог.
- 5 Сравниваем итог с порогом.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
price = 120
quantity = 3
total = price * quantity
is_expensive = total > 300
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
price = 120
quantity = 3
total = price * quantity
is_expensive = total > 300

print(total)
print(is_expensive)
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

360
True

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Создай карточку пользователя: name, age, rating и is_active. Выведи значения и их типы.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Используй `str`, `int`, `float` и `bool`; `type()` покажет реальный тип значения.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
name = "Alex"  
age = 28  
rating = 4.8  
is_active = True  
for value in [name, age, rating, is_active]:  
    print(value, type(value).__name__)
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

Alex str
28 int
4.8 float
True bool

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую законченную задачу.

ЗАДАНИЕ

Собери мини-чек: цена, количество, скидка и итог после скидки.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.

A dark-themed terminal window with a title bar at the top containing three colored circles (red, yellow, green) on the left and the filename 'mini_project' on the right. The terminal contains a Python script for calculating a total price with a discount.

```
price = 200
quantity = 2
discount = 0.10
subtotal = price * quantity
total = subtotal * (1 - discount)
print(total)
```

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

360.0

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Давать переменным непонятные имена.

02

Путать = и сравнение ==.

03

Складывать несовместимые типы.

04

Преобразовывать данные, не проверяя их содержимое.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «variable» и какую проблему это понятие помогает описать?

2

Чем «variable» отличается от «str»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **variable** — имя, связанное со значением в программе.

2 variable: имя, связанное со значением в программе. str: тип текстовых строк.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3

Анна
25
True

4

"25" — строка, а 1 — число. Python не складывает их напрямую. Сначала строку нужно преобразовать в int.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Уверенно хранить текст, числа и логические значения в переменных и осознанно преобразовывать типы.
- ✓ Читать и объяснять пример темы «Переменные и типы данных».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Условия и логика

Завтра: Условия и логика. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.